

字节跳动与阿里前端面经

字节前端面试真题

一面

1. 实现一个深拷贝函数，如何处理循环引用？
2. 浏览器缓存策略有哪些？优先级顺序是怎样的？
3. 实现Promise.all的polyfill
4. 用CSS实现垂直水平居中，至少写出三种方案
5. 说说Event Loop在浏览器和Node.js中的差异
6. 如何实现页面秒开率从65%提升到85%？

二面

1. 手写函数柯里化实现 $\text{add}(1)(2)(3) = 6$
2. 实现虚拟DOM的diff算法需要考虑哪些边界情况？
3. Webpack的Tree Shaking原理及实现条件
4. 如何设计一个前端监控系统？
5. 实现async/await的generator polyfill
6. 解释React Hooks的闭包陷阱及解决方案
7. 设计一个多层级菜单组件，如何实现千万级节点的高性能渲染？
8. 前端安全防护方案有哪些？XSS和CSRF的防御手段
9. 微前端方案qiankun的沙箱隔离原理
10. 服务端渲染hydration过程可能出现的问题
11. 实现一个支持撤销重做的状态管理系统

参考答案

一面参考答案：

1. 实现一个深拷贝函数，如何处理循环引用？

思路:

1. 创建一个 `WeakMap` 来存储已经拷贝过的对象 (`originalObject` -> `copiedObject`)。
2. 在拷贝函数内部, 首先检查当前要拷贝的对象是否已经存在于 `WeakMap` 中。
 - 如果存在, 直接返回 `WeakMap` 中对应的拷贝后的对象。
 - 如果不存在, 继续拷贝。
3. 对于对象和数组, 创建一个新的空对象或空数组。
4. 将新创建的拷贝对象存入 `WeakMap` (key 是原始对象, value 是拷贝对象)。
5. 递归地拷贝原始对象的所有属性 (对于对象) 或元素 (对于数组) 到新的拷贝对象中。

示例代码 (JavaScript):

代码块

```
1  function deepClone(obj, hash = new WeakMap()) {
2    if (obj === null || typeof obj !== 'object') {
3      return obj; // 基本类型或null直接返回
4    }
5
6    if (hash.has(obj)) {
7      return hash.get(obj); // 处理循环引用
8    }
9
10   let clone;
11   if (Array.isArray(obj)) {
12     clone = [];
13   } else if (obj instanceof Date) {
14     clone = new Date(obj);
15   } else if (obj instanceof RegExp) {
16     clone = new RegExp(obj.source, obj.flags);
17   } else if (obj.constructor && typeof obj.constructor === 'function') {
18     // 尝试保留原型链, 但对于复杂对象可能不完美
19     // 注意: 这不处理构造函数参数, 对于内置对象如 Map, Set 等需要额外处理
20     clone = Object.create(Object.getPrototypeOf(obj));
21   } else {
22     clone = {};
23   }
24
25   hash.set(obj, clone); // 存储已拷贝的对象
26
27   for (let key in obj) {
28     if (Object.prototype.hasOwnProperty.call(obj, key)) {
29       clone[key] = deepClone(obj[key], hash);
30     }
31   }
32 }
```

```

30     }
31 }
32
33 // 如果需要拷贝 Symbol 属性
34 // const symbols = Object.getOwnPropertySymbols(obj);
35 // for (let symbol of symbols) {
36 //   clone[symbol] = deepClone(obj[symbol], hash);
37 // }
38
39 return clone;
40 }
41
42 // 测试循环引用
43 const a = { name: 'a' };
44 const b = { name: 'b' };
45 a.ref = b;
46 b.ref = a;
47
48 const c = deepClone(a);
49 console.log(c.name); // a
50 console.log(c.ref.name); // b
51 console.log(c.ref.ref === c); // true, 循环引用被正确处理

```

关键点: `WeakMap` 的使用。`WeakMap` 的键是弱引用的, 这意味着当原始对象没有其他引用时, 它可以被垃圾回收, `WeakMap` 中的条目也会随之消失, 不会造成内存泄漏。

2. 浏览器缓存策略有哪些? 优先级顺序是怎样的?

浏览器缓存策略主要分为强缓存 (Strong Caching) 和协商缓存 (Negotiation Caching)。它们的目的是减少网络请求, 加快资源加载速度。

缓存策略类型:

1. 强缓存: 浏览器直接从本地缓存中读取资源, 不发送任何 HTTP 请求到服务器。

- `Expires` (HTTP/1.0): 指定一个绝对的过期时间。如果当前时间在此时间之前, 缓存有效。缺点是依赖客户端时间, 可能不准确。
- `Cache-Control` (HTTP/1.1): 更强大和灵活。常用指令:
 - `public`: 资源可以被任何缓存 (包括 CDN、代理服务器、浏览器) 缓存。
 - `private`: 资源只能被最终用户的浏览器缓存。
 - `no-cache`: 需要进行协商缓存。浏览器在使用缓存副本前, 必须与服务器确认资源是否已更改。它并不是完全不使用缓存, 而是先验证。

- `no-store` : 完全禁止缓存。浏览器不存储响应，每次都从服务器请求完整资源。
- `max-age=<seconds>` : 指定资源在被视为“新鲜”状态的最长时间 (相对时间，从响应生成开始计算)。这是强缓存的核心。
- `s-maxage=<seconds>` : 类似于 `max-age`，但仅应用于共享缓存 (如 CDN)。
- `must-revalidate` : 一旦资源过期 (例如 `max-age` 过期)，在成功向原始服务器验证之前，不能使用该资源的旧副本。

2. 协商缓存 (也叫条件缓存): 强缓存失效后 (或 `Cache-Control: no-cache` 时)，浏览器会向服务器发送一个 HTTP 请求，但这个请求会带上一些上次缓存时的标识信息。服务器根据这些信息判断资源是否有更新。

◦ `Last-Modified` / `If-Modified-Since` :

- `Last-Modified` (服务器响应头): 资源在服务器上最后一次修改的时间。
- `If-Modified-Since` (浏览器请求头): 浏览器上次缓存该资源时，服务器返回的 `Last-Modified` 值。服务器比较这个时间和资源的实际最后修改时间。
 - 如果未修改，服务器返回 `304 Not Modified` 状态码，响应体为空，浏览器直接使用本地缓存。
 - 如果已修改，服务器返回 `200 OK` 和新的资源内容。
- 缺点：时间戳精度问题 (只能到秒)；文件内容没变但修改时间变了也可能导致重新下载。

◦ `ETag` / `If-None-Match` :

- `ETag` (Entity Tag, 服务器响应头): 资源内容的唯一标识符 (类似指纹)，通常是内容的哈希值。只要内容改变，`ETag` 就会改变。
- `If-None-Match` (浏览器请求头): 浏览器上次缓存该资源时，服务器返回的 `ETag` 值。服务器比较这个 `ETag` 和当前资源的 `ETag`。
 - 如果匹配 (未修改)，服务器返回 `304 Not Modified`。
 - 如果不匹配 (已修改)，服务器返回 `200 OK` 和新的资源内容及新的 `ETag`。
- 优点：比 `Last-Modified` 更精确，能解决 `Last-Modified` 的一些问题。

优先级顺序:

当浏览器请求一个资源时，会按照以下大致顺序检查缓存：

1. `Cache-Control: no-store` : 如果存在，直接跳过所有缓存，向服务器请求全新资源。

2. 强缓存检查:

- `Cache-Control: max-age` (HTTP/1.1): 如果存在且未过期，直接使用缓存，不发请求。这是最高优先级的强缓存策略。
- `Expires` (HTTP/1.0): 如果 `Cache-Control: max-age` 不存在或已过期 (或浏览器不支持 HTTP/1.1)，则检查 `Expires`。如果存在且未过期，直接使用缓存。

- (注意: 如果 `Cache-Control` 和 `Expires` 同时存在, `Cache-Control` 的优先级更高。)
3. 协商缓存检查 (当强缓存失效或 `Cache-Control: no-cache` 时):
- 浏览器发送请求到服务器。
 - `If-None-Match` (配合 `ETag`): 如果上一次响应中有 `ETag`, 浏览器会在请求头带上 `If-None-Match`。服务器比较 `ETag`。
 - `If-Modified-Since` (配合 `Last-Modified`): 如果上一次响应中有 `Last-Modified` 且没有 `ETag` (或者服务器不支持 `ETag`), 浏览器会在请求头带上 `If-Modified-Since`。服务器比较时间。
 - (注意: 如果服务器同时支持 `ETag` 和 `Last-Modified`, `ETag` 的优先级通常更高, 因为它更精确。)
 - 如果协商缓存命中 (服务器返回 `304 Not Modified`), 浏览器使用本地缓存。
 - 如果协商缓存未命中 (服务器返回 `200 OK` 和新资源), 浏览器下载新资源并更新缓存。
4. `Cache-Control: no-cache`: 即使本地有缓存副本, 也必须先向服务器验证 (即执行协商缓存)。
5. 无缓存策略或缓存过期: 如果以上策略都不适用或缓存已过期且协商缓存未命中, 浏览器会从服务器请求完整的资源。

总结优先级:

`Cache-Control (no-store)` > `Cache-Control (max-age/s-maxage)` > `Expires`
> `ETag + If-None-Match` > `Last-Modified + If-Modified-Since`

其中 `no-cache` 强制进入协商缓存阶段。

3. 实现Promise.all的polyfill

`Promise.all(iterable)` 方法返回一个 `Promise` 对象, 它在 `iterable` 参数内所有的 `Promise` 都变为成功resolved状态后才变为成功resolved状态, 或者在 `iterable` 参数内任意一个 `Promise` 变为失败rejected状态后立即变为失败rejected状态。

思路:

1. `Promise.all` 接收一个可迭代对象 (如数组) 作为参数。
2. 返回一个新的 `Promise`。
3. 遍历可迭代对象中的每一个元素:
 - 如果元素不是 `Promise`, 将其包装成一个立即 resolved 的 `Promise`。
 - 监听每个 `Promise` 的状态。

4. 维护一个计数器，记录已 resolved 的 `Promise` 数量，以及一个数组来存储所有 `Promise` 的 resolved 值 (并保持原始顺序)。
5. 如果所有 `Promise` 都成功 resolved，则新的 `Promise` 以结果数组 resolved。
6. 如果任何一个 `Promise` 被 rejected，则新的 `Promise` 立即以该 `Promise` 的 rejection 原因 rejected，无需等待其他 `Promise`。
7. 处理空的可迭代对象：如果传入的是空数组，`Promise.all` 应该立即 resolve 为一个空数组。

示例代码 (JavaScript):

代码块

```
1  function MyPromiseAll(promises) {
2    return new Promise((resolve, reject) => {
3      if (!Array.isArray(promises)) {
4        return reject(new TypeError('Argument must be an array.'));
5      }
6
7      const results = [];
8      let resolvedCount = 0;
9      const numPromises = promises.length;
10
11     if (numPromises === 0) {
12       resolve([]); // 空数组直接 resolve
13       return;
14     }
15
16     promises.forEach((promise, index) => {
17       Promise.resolve(promise) // 确保每个元素都是 Promise
18         .then(value => {
19           results[index] = value; // 按原始顺序存储结果
20           resolvedCount++;
21
22           if (resolvedCount === numPromises) {
23             resolve(results);
24           }
25         })
26         .catch(error => {
27           reject(error); // 任何一个 promise 失败，整体就失败
28         });
29     });
30   });
31 }
32
33 // 测试用例
```

```

34  const p1 = Promise.resolve(1);
35  const p2 = new Promise((resolve) => setTimeout(() => resolve(2), 100));
36  const p3 = 3; // 非 Promise 值
37  const p4Error = Promise.reject('Error in p4');
38  const p5 = new Promise((resolve) => setTimeout(() => resolve(5), 50));
39
40  console.log("Test Case 1: All resolve");
41  MyPromiseAll([p1, p2, p3, p5])
42    .then(results => console.log("All resolved:", results)) // Expected: [1, 2,
    3, 5]
43    .catch(error => console.error("Error:", error));
44
45  console.log("\nTest Case 2: One rejects");
46  MyPromiseAll([p1, p2, p4Error, p5])
47    .then(results => console.log("All resolved:", results))
48    .catch(error => console.error("One rejected:", error)); // Expected: "Error
    in p4"
49
50  console.log("\nTest Case 3: Empty array");
51  MyPromiseAll([])
52    .then(results => console.log("Empty array resolved:", results)) // Expected:
    []
53    .catch(error => console.error("Error with empty array:", error));
54
55  const pDelayedError = new Promise((resolve, reject) => setTimeout(() =>
    reject('Delayed Error'), 50));
56  const pDelayedSuccess = new Promise((resolve) => setTimeout(() =>
    resolve('Delayed Success'), 100));
57
58  console.log("\nTest Case 4: Error happens before all resolve");
59  MyPromiseAll([pDelayedSuccess, pDelayedError])
60    .then(results => console.log("All resolved:", results))
61    .catch(error => console.error("Error before all resolve:", error)); //
    Expected: "Delayed Error"

```

关键点:

- 使用 `Promise.resolve()` 来处理非 `Promise` 的值。
- 正确处理结果的顺序。
- 一旦有 `Promise` 失败, 立即 `reject`。
- 处理空数组的边界情况。

4. 用CSS实现垂直水平居中, 至少写出三种方案

有多种方法可以使用 CSS 实现元素的垂直水平居中。以下是几种常见的方案：

场景假设: 我们要居中一个 `div` (子元素) 在其父容器 `div` (父元素) 中。

代码块

```
1 <div class="parent">
2   <div class="child">
3     Centered Content
4   </div>
5 </div>
```

方案一: Flexbox (推荐)

这是现代 CSS 中最简单、最灵活的居中方案。

代码块

```
1 .parent {
2   display: flex;
3   justify-content: center; /* 水平居中 */
4   align-items: center;    /* 垂直居中 */
5   /* 需要给父元素一个高度才能看到垂直居中效果 */
6   height: 300px;
7   border: 1px solid black;
8 }
9
10 .child {
11   width: 100px;
12   height: 50px;
13   background-color: lightblue;
14 }
```

方案二: Grid (同样推荐)

Grid布局也提供了非常直观的居中方式。

代码块

```
1 .parent {
2   display: grid;
3   place-items: center; /* 水平垂直居中的简写 */
4   /* 或者分开写:
```



```
5    justify-items: center;
6    align-items: center;
7    如果子元素只有一个，也可以用 justify-content 和 align-content */
8    height: 300px;
9    border: 1px solid black;
10 }
11
12 .child {
13     width: 100px;
14     height: 50px;
15     background-color: lightcoral;
16 }
```

或者针对子元素使用：

代码块

```
1  .parent {
2      display: grid;
3      height: 300px;
4      border: 1px solid black;
5  }
6  .child {
7      justify-self: center; /* 在网格区域内水平居中 */
8      align-self: center; /* 在网格区域内垂直居中 */
9      width: 100px;
10     height: 50px;
11     background-color: lightcoral;
12 }
```

方案三:绝对定位 (Absolute Positioning) + Transform

这种方法适用于子元素尺寸未知或已知的情况。

代码块

```
1  .parent {
2      position: relative; /* 父元素需要相对定位 */
3      height: 300px;
4      border: 1px solid black;
5  }
6
7  .child {
```

```
8     position: absolute;
9     top: 50%;
10    left: 50%;
11    transform: translate(-50%, -50%); /* 根据自身宽高进行偏移 */
12    width: 100px; /* 宽度和高度可选，如果未知也能工作 */
13    height: 50px;
14    background-color: lightgreen;
15 }
```

方案四: 绝对定位 + auto margins (子元素尺寸已知)

如果子元素的尺寸是固定的，可以使用这种方法。

代码块

```
1  .parent {
2      position: relative;
3      height: 300px;
4      border: 1px solid black;
5  }
6
7  .child {
8      position: absolute;
9      top: 0;
10     bottom: 0;
11     left: 0;
12     right: 0;
13     margin: auto; /* 关键 */
14     width: 100px; /* 必须指定宽高 */
15     height: 50px; /* 必须指定宽高 */
16     background-color: gold;
17 }
```

方案五: Line Height (单行文本或行内块元素垂直居中)

这种方法主要用于单行文本或行内/行内块元素的垂直居中。

代码块

```
1  .parent {
2      height: 300px;
3      line-height: 300px; /* 使行高等于容器高度 */
4      text-align: center; /* 水平居中行内内容 */
5      border: 1px solid black;
6      font-size: 0; /* 可选，消除行内块间隙 */
7  }
```

```
7   }
8
9   .child {
10    display: inline-block; /* 或 inline */
11    vertical-align: middle; /* 垂直居中 */
12    line-height: normal; /* 重置子元素的行高 */
13    width: 100px;
14    height: auto; /* 高度可以自适应内容, 或固定 */
15    padding: 10px;
16    background-color: violet;
17    font-size: 16px; /* 恢复字体大小 */
18  }
```

对于 `div` 这种块级元素，可以将其 `display` 设置为 `inline-block` 然后应用 `vertical-align: middle`。

5. 说说Event Loop在浏览器和Node.js中的差异

事件循环 (Event Loop) 是 JavaScript 实现异步编程的核心机制。虽然浏览器和 Node.js 都使用事件循环，但它们的实现和具体行为存在一些关键差异，主要源于它们运行环境的不同。

共同点:

- 都有一个调用栈 (Call Stack) 用于执行同步代码。
- 都有一个消息队列 (Message Queue / Task Queue) 用于存放异步任务的回调。
- 事件循环不断从消息队列中取出任务并放入调用栈执行。

主要差异:

1. 运行环境:

- 浏览器: 事件循环与 Web API (如 `setTimeout`, `DOM` 事件, `AJAX` 请求) 交互。这些 API 由浏览器内核的不同模块 (渲染引擎、JavaScript 引擎、HTTP 请求模块等) 提供。
- Node.js: 事件循环与 Node.js 内置的 C++ API (如 `fs` 模块进行文件操作, `http` 模块创建服务器, `timers` 模块) 交互, 底层依赖 `libuv` 库来处理异步 I/O。

2. 任务队列的种类和处理:

- 浏览器: 通常至少有两种任务队列:
 - 宏任务 (Macrotask / Task): 包括 `script` (整体代码块)、`setTimeout`, `setInterval`, `setImmediate` (非标准, IE 和 Node.js 支持), I/O, UI rendering。每次事件循环迭代只会从宏任务队列中取出一个任务执行。

- 微任务 (Microtask / Job): 包括 `Promise.then/catch/finally`, `MutationObserver`, `queueMicrotask()` (较新的 API)。微任务在当前宏任务执行完毕后、下一次宏任务开始前, 会立即全部执行。如果微任务执行过程中又产生了新的微任务, 这些新的微任务也会在当前微任务队列清空前执行。

- Node.js:

- Node.js 的事件循环分为多个阶段 (Phases), 每个阶段都有自己的 FIFO 队列来执行回调。
- 微任务队列 (`process.nextTick()` 和 `Promise.then/catch/finally`):
 - `process.nextTick()` 的回调优先级极高, 它会在当前操作完成后、事件循环的任何其他阶段 (包括其他微任务) 之前立即执行。
 - `Promise` 的微任务会在当前操作完成后, 但在 `process.nextTick()` 队列清空之后, 以及事件循环进入下一个阶段之前执行。
- 宏任务队列 (按阶段划分): Node.js 事件循环的主要阶段大致如下 (libuv 实现):
 1. `timers` 阶段: 执行 `setTimeout()` 和 `setInterval()` 设置的回调。
 2. `pending callbacks` (I/O callbacks / DEPRECATED in docs, but relevant for understanding): 执行延迟到下一个循环迭代的 I/O 回调 (例如, 某些系统操作的错误回调)。
 3. `idle, prepare` 阶段: 仅内部使用。
 4. `poll` 阶段 (轮询阶段): 检索新的 I/O 事件; 执行与 I/O 相关的回调 (几乎所有回调, 除了 `close` 回调、`timers` 和 `setImmediate`)。如果 `poll` 队列为空:
 - 如果有 `setImmediate()` 回调, 则进入 `check` 阶段。
 - 如果没有 `setImmediate()` 回调, 且有 `timers` 即将到期, 则等待 `timers` 到期后返回 `timers` 阶段。
 - 否则, 会阻塞并等待新的 I/O 事件。
 5. `check` 阶段: 执行 `setImmediate()` 的回调。
 6. `close callbacks` 阶段: 执行一些关闭回调, 如 `socket.on('close', ...)`。
- 微任务执行时机: 在 Node.js 中, 微任务 (`process.nextTick()` 和 `Promise` 回调) 会在每个阶段完成后以及回调之间被检查并执行。这意味着在一个阶段内执行一个宏任务回调后, 会立即清空微任务队列, 然后再执行该阶段的下一个宏任务回调 (如果还有的话), 或者进入下一个阶段。

3. `setTimeout` 和 `setImmediate` 的行为:

- 浏览器: `setImmediate` 不是标准 API, 主要在 IE 和 Node.js 中存在。
- Node.js:
 - `setTimeout(fn, 0)`: 其回调会在 `timers` 阶段执行。

- `setImmediate(fn)` : 其回调会在 `check` 阶段执行。
- 当 `setTimeout(fn, 0)` 和 `setImmediate(fn)` 在同一个 I/O 周期内 (例如, 在一个模块的顶层作用域或一个已完成的 I/O 回调中) 被调用时, 它们的执行顺序是不确定的, 取决于进程性能和系统调用耗时。
- 但是, 如果它们在一个 I/O 回调 (例如 `fs.readFile` 的回调) 中被调用, `setImmediate` 会先于任何 `setTimeout` 执行, 因为 I/O 回调属于 `poll` 阶段, `poll` 阶段之后是 `check` 阶段, 然后才是下一个循环的 `timers` 阶段。

4. `process.nextTick()` (Node.js 特有):

- 它不属于事件循环的任何特定阶段, 而是独立于事件循环的“快速通道”。
- `process.nextTick()` 的回调会在当前 JavaScript 操作 (例如当前同步代码块或当前宏任务回调) 执行完毕后、事件循环继续到下一个阶段或下一个宏任务之前立即执行。
- 它的优先级高于 `Promise` 的微任务。如果递归调用 `process.nextTick()`, 可能会阻塞事件循环, 导致 I/O "饿死"。

总结差异表格:

特性	浏览器环境	Node.js 环境 (libuv)
核心API	Web APIs (DOM, Workspace, setTimeout)	Node.js 内置模块 (fs, http, timers), libuv
宏任务队列	单个宏任务队列 (通常)	多个阶段, 每个阶段有自己的 FIFO 队列
微任务处理	当前宏任务后, 渲染前, 清空整个微任务队列	每个阶段完成后, 以及回调之间, 清空微任务队列
<code>process.nextTick()</code>	不存在	存在, 优先级高于 Promise 微任务, 在当前操作后立即执行
<code>setImmediate()</code>	非标准 (IE支持)	在 check 阶段执行, 通常在 I/O 回调后优先于 <code>setTimeout(fn,0)</code>
主要目标	响应用户交互, 渲染UI, 处理网络请求	高并发 I/O, 服务器端逻辑

理解这些差异对于编写高效的、行为可预测的异步 JavaScript 代码至关重要，尤其是在涉及到跨环境兼容性或性能调优时。

6. 如何实现页面秒开率从65%提升到85%?

将页面秒开率 (通常指页面在 1 秒或 2 秒内完成关键内容加载并可交互的比例) 从 65% 提升到 85% 是一个显著的性能优化目标，需要多方面的综合施策。以下是一些关键策略和步骤：

一、精确测量与分析 (Measure & Analyze First)

1. 定义“秒开”标准: 明确“秒开”具体指标，例如：

- FCP (First Contentful Paint): 首次绘制任何文本、图像、非空白 canvas 或 SVG 的时间。
- LCP (Largest Contentful Paint): 视口中最大可见内容元素完成渲染的时间。
- TTI (Time to Interactive): 页面完全加载并且能够可靠地快速响应用户输入的时间。
- Speed Index: 页面内容填充的平均速度。
- 通常，LCP 或 FCP $\leq$ 1s (或 2s, 根据业务要求) 可以作为“秒开”的衡量标准。

2. 选择合适的工具:

- Lighthouse (Chrome DevTools): 提供全面的性能审计和改进建议。
- WebPageTest: 在真实设备和网络条件下进行详细测试。
- Google PageSpeed Insights: 结合了 CrUX (Chrome User Experience Report) 真实用户数据和 Lighthouse 分析。
- Performance Tab (Chrome DevTools): 深入分析加载过程中的火焰图、网络请求、JS 执行等。
- 真实用户监控 (RUM - Real User Monitoring): 收集实际用户数据，了解不同网络、设备下的真实性能瓶颈。例如 Sentry, Datadog, New Relic 或自建系统。

3. 识别瓶颈: 通过工具分析，找出拖慢页面加载的关键因素：

- 网络延迟: DNS 查询慢、TCP 连接慢、TTFB (Time To First Byte) 长。
- 资源大小: 图片过大、JS/CSS 文件过大。
- 资源数量: HTTP 请求过多。
- 渲染阻塞: CSS 和 JS 阻塞页面渲染。
- JavaScript 执行: 长时间运行的 JS 脚本、复杂的框架初始化。
- 服务器性能: 后端处理慢。

二、优化策略 (Optimization Strategies)

A. 减少请求大小和数量 (Reduce Request Size & Count)

1. 资源压缩:

- 文本资源 (HTML, CSS, JS): 使用 Gzip 或 Brotli 压缩。Brotli 通常有更好的压缩率。
- 图片优化:
 - 选择合适的格式: WebP > AVIF > JPEG/PNG。
 - 图片压缩: 使用工具如 TinyPNG/JPEGoptim/ImageOptim。
 - 响应式图片: 使用 `<picture>` 元素或 `srcset` 属性提供不同尺寸的图片。
- 字体优化:
 - 使用 WOFF2 格式。
 - 字体子集化 (Font Subsetting): 只包含页面用到的字符。

2. 代码分割 (Code Splitting):

- 只加载当前页面或用户操作所需的 JS 和 CSS。利用 Webpack, Rollup, Parcel 等构建工具的动态 `import()`。

3. 合并文件 (Bundling):

- 在 HTTP/1.1 下, 合并 JS 和 CSS 文件以减少请求数。
- 在 HTTP/2 和 HTTP/3 下, 由于多路复用, 合并文件的收益降低, 但仍需适度, 避免单个文件过大。

4. 移除未使用的代码 (Tree Shaking & Dead Code Elimination):

- 使用构建工具移除 JS 和 CSS 中未被引用的代码。

5. 使用 CDN (Content Delivery Network):

- 将静态资源部署到 CDN, 利用其地理位置优势, 减少用户访问延迟, 提高可用性。

B. 优化关键渲染路径 (Optimize Critical Rendering Path)

1. 减少渲染阻塞资源:

- CSS:
 - 将关键 CSS (Critical CSS) 内联到 `<head>` 中, 以尽快渲染首屏内容。
 - 异步加载非关键 CSS: `<link rel="stylesheet" href="styles.css" media="print" onload="this.media='all'">` 或使用 `loadCSS` 库。
- JavaScript:
 - 将 `<script>` 标签放在 `<body>` 底部。
 - 使用 `async` 或 `defer` 属性加载 JS:
 - `async`: 异步下载, 下载完成后立即执行, 可能阻塞 HTML 解析 (如果解析未完成)。

- `defer`: 异步下载, 在 HTML 解析完成后、`DOMContentLoaded` 事件前按顺序执行。通常更推荐。

2. 预加载/预连接/预获取:

- `dns-prefetch`: `<link rel="dns-prefetch" href="//example.com">` 提前解析 DNS。
- `preconnect`: `<link rel="preconnect" href="//example.com">` 提前建立 TCP 连接 (+TLS)。
- `preload`: `<link rel="preload" href="critical.js" as="script">` 提前加载当前导航会用到的关键资源, 优先级高。
- `prefetch`: `<link rel="prefetch" href="next-page.js" as="script">` 提前加载未来导航可能用到的资源, 优先级低。

3. 避免强制同步布局 (Layout Thrashing):

- 在 JS 中批量读取 DOM 属性, 然后批量写入, 避免读写交错导致多次重排/重绘。

C. 优化 JavaScript 执行 (Optimize JavaScript Execution)

1. 减少主线程工作量:

- 将耗时计算或非 UI 相关任务移到 Web Workers。
- 优化算法和数据结构。
- 避免长时间运行的脚本。

2. 框架和库优化:

- 选择轻量级框架/库, 或按需加载其模块。
- 了解所用框架的性能最佳实践 (如 React 的 `memo`, `useCallback`, `useMemo`, 虚拟列表)。

3. 懒加载 (Lazy Loading):

- 图片懒加载: 仅当图片进入视口时才加载 (使用 `IntersectionObserver` API 或 `loading="lazy"` 属性)。
- 组件/模块懒加载: 结合代码分割, 按需加载非首屏组件。

D. 服务器端和网络优化 (Server-Side & Network Optimization)

1. 提升 TTFB (Time To First Byte):

- 优化后端代码和数据库查询。
- 使用更快的服务器硬件或云服务。
- 启用服务器端缓存 (如 Redis, Memcached)。
- 使用 HTTP/2 或 HTTP/3 协议。

2. 浏览器缓存策略: (如问题2所述)

- 合理配置 `Cache-Control`，`ETag`，`Last-Modified`，充分利用强缓存和协商缓存。

3. Service Workers:

- 实现离线缓存，缓存静态资源甚至 API 请求，可以极大提升二次访问速度，甚至在弱网/离线时可用。

E. 用户感知性能优化 (Optimize Perceived Performance)

1. 骨架屏 (Skeleton Screens): 在内容加载完成前显示页面的大致结构，减少用户等待焦虑。
2. 占位符 (Placeholders): 为图片和内容区域提供占位。
3. 渐进式加载 (Progressive Loading): 先显示基本内容，再逐步加载增强功能。

实施与迭代:

1. 设定优先级: 根据瓶颈分析结果，优先处理影响最大的问题。
2. 小步快跑，持续监控: 每次优化后，都要进行测试和数据对比，验证效果。
3. A/B 测试: 对于某些不确定的优化，可以进行 A/B 测试，看其对核心业务指标和用户体验的影响。
4. 建立性能预算 (Performance Budget): 为关键指标 (如 LCP, JS 大小) 设定上限，防止性能退化。

通过系统性地应用上述策略，并持续监控和迭代，将页面秒开率从 65% 提升到 85% 是完全可能实现的。关键在于数据驱动和持续优化。

二面参考答案:

1. 手写函数柯里化实现 `add(1)(2)(3) = 6`

函数柯里化 (Currying) 是一种将接受多个参数的函数转换成一系列只接受单个参数函数的技术。

思路:

我们需要一个函数 `add`，它第一次调用时可以接受一个参数，并返回一个新的函数。这个新的函数又可以接受下一个参数，并返回再下一个新的函数，以此类推。当不再有参数传入（或者达到预期的参数数量时），它就执行实际的加法运算。

为了实现 `add(1)(2)(3) = 6` 这种链式调用，并且在最后能得到结果，我们可以让函数在被调用时，如果后面没有再继续调用（即被当作值使用时，例如参与运算或打印），则执行求和。这通常通过重写函数的 `toString` 或 `valueOf` 方法来实现，或者通过一个显式的执行函数调用。

一个更通用的柯里化实现是收集所有参数，直到参数数量足够，或者通过一个特殊调用来触发计算。对于 `add(1)(2)(3)` 这种形式，我们希望它能无限接收参数，直到某一次调用不传参数时才返回值，或者像题目中那样，它本身能够进行隐式转换得到值。

这里提供一个通过闭包和函数递归调用的实现，当函数被尝试转换为原始值时（例如进行数学运算），会触发 `valueOf` 方法。

代码块

```
1  function add(x) {
2    let sum = x;
3
4    function curriedAdd(y) {
5      sum += y;
6      return curriedAdd; // 返回自身，以便继续链式调用
7    }
8
9    // 当函数被当作原始值使用时（例如参与运算或隐式转换），valueOf 会被调用
10   curriedAdd.valueOf = function() {
11     return sum;
12   };
13
14   // toString 也可以类似实现，确保在不同上下文下都能正确返回值
15   curriedAdd.toString = function() {
16     return sum.toString();
17   };
18
19   return curriedAdd;
20 }
21
22 console.log(add(1)(2)(3) == 6); // true
23 console.log(add(1)(2)(3)(4) == 10); // true
24
25 const a1 = add(1);
26 const a2 = a1(2);
27 const a3 = a2(3);
28 console.log(a3 == 6); // true
29 console.log(+a3); // 6 (显式转换为数字)
```

解释:

1. `add(x)`: 初始调用，接收第一个参数 `x` 并初始化 `sum`。
2. `curriedAdd(y)`: 内部函数，每次调用会累加参数 `y` 到 `sum` 上。关键在于它返回自身 (`curriedAdd`)，这使得链式调用 (`(val)(val)`) 成为可能。
3. `curriedAdd.valueOf`: 当 `curriedAdd` 的实例（即 `add(1)` 或 `add(1)(2)` 返回的函数）被期望转换为一个原始值时（例如在 `==` 比较或 `+` 一元操作符中），JavaScript 会自动调用其 `valueOf` 方法。我们重写此方法使其返回当前的 `sum`。
4. `curriedAdd.toString`: 类似 `valueOf`，当需要字符串表示时调用。

这种实现依赖于 JavaScript 的隐式类型转换来触发 `valueOf`。

另一种常见的柯里化实现 (固定参数数量):

如果我们需要一个更通用的柯里化函数, 它接收一个目标函数和该函数期望的参数个数:

代码块

```
1  function curry(fn, arity = fn.length) {
2    return function curried(...args) {
3      if (args.length >= arity) {
4        return fn.apply(this, args);
5      } else {
6        return function(...nextArgs) {
7          return curried.apply(this, args.concat(nextArgs));
8        };
9      }
10   };
11 }
12
13 // 使用示例
14 function sumThree(a, b, c) {
15   return a + b + c;
16 }
17
18 const curriedSumThree = curry(sumThree);
19 console.log(curriedSumThree(1)(2)(3)); // 6
20 console.log(curriedSumThree(1, 2)(3)); // 6
21 console.log(curriedSumThree(1)(2, 3)); // 6
```

这更接近柯里化的标准定义。题目中的 `add(1)(2)(3)` 是一个特例, 通常要求在最后能“感知”到调用结束。

2. 实现虚拟DOM的diff算法需要考虑哪些边界情况?

虚拟 DOM (Virtual DOM) 的 diff 算法是现代前端框架 (如 React, Vue) 的核心之一, 用于高效地更新真实 DOM。在实现 diff 算法时, 需要考虑以下边界情况以确保其正确性和性能:

1. 节点类型改变:

- 如果新旧节点的 `type` (例如从 `div` 变成 `p`, 或者从自定义组件 `A` 变成 `B`) 不同, 通常会直接卸载旧节点及其所有子节点, 然后挂载新节点。不需要再进行子节点的 diff。
- 边界: 文本节点和元素节点的转换。

2. Key 的重要性与处理:

- 无 Key 或 Key 重复: 当比较一组子节点 (列表) 时, 如果没有唯一的 `key`, diff 算法可能进行低效的更新 (例如, 在列表头部插入一个元素, 可能导致所有后续元素被认为是修改, 而不是简单地移动和插入)。
- Key 的改变: 如果一个元素的 `key` 改变了, 即使其他属性和子节点都相同, 它也会被视为一个全新的元素, 导致旧元素卸载, 新元素挂载。
- Key 的顺序变化: 在列表项重排时, 通过 `key` 可以高效地识别出是移动操作, 而不是删除旧元素并创建新元素。diff 算法需要正确处理节点的移动。
- 边界: `null` 或 `undefined` 作为 key 值。

3. 仅属性变化:

- 节点类型和 key 相同, 但属性 (props/attributes) 发生变化 (如 `className`, `style`, `id` 等)。diff 算法应仅更新变化的属性, 避免重新渲染整个节点。
- 边界: `style` 对象的深度比较 vs 浅比较; 事件处理函数的更新 (通常是替换)。`value` 和 `checked` 属性的特殊处理。

4. 仅文本内容变化:

- 对于文本节点, 如果内容发生变化, 直接更新其文本内容。
- 边界: 元素节点内部从文本内容变成子元素, 或反之。

5. 子节点列表的变化:

- 新增节点: 在列表末尾、开头或中间插入新节点。
- 删除节点: 从列表末尾、开头或中间删除节点。
- 节点重排: 列表项顺序改变。这是 `key` 最能发挥作用的地方。
- 混合操作: 同时发生增、删、改、移。
- 边界: 空列表变为有内容的列表, 或反之; 列表中包含 `null`、`undefined` 或 `false` (在 React 中通常表示不渲染任何东西)。

6. 条件渲染 (Conditional Rendering):

- 一个节点之前存在, 现在不存在了 (例如 `condition && <Component />`, `condition` 从 `true` 变为 `false`)。
- 一个节点之前不存在, 现在存在了。
- 边界: 由条件渲染导致的节点类型变化。

7. Fragment (片段):

- 当使用 Fragment (`<React.Fragment>` 或 `<>...</>`) 时, 它本身不对应真实 DOM 节点, diff 算法需要能正确处理其子节点的 diff 和挂载。
- 边界: Fragment 的子节点直接成为父节点的子节点, 层级关系的处理。

8. 组件的生命周期与状态:

- 对于组件节点，diff 算法不仅比较 props，还需要考虑组件自身的更新逻辑。
- 当组件更新时，diff 算法会递归地对其 `render` 方法返回的虚拟 DOM 进行 diff。
- 边界：`shouldComponentUpdate` (类组件) 或 `React.memo` (函数组件) 返回 `false`，阻止了子树的 diff。

9. SVG 和其他命名空间:

- 创建和更新 SVG 元素时，需要使用正确的命名空间 (`createElementNS`)。diff 算法需要能识别并处理这种情况。

10. 输入框等表单元素的值:

- 对于受控组件，如输入框的 `value`，需要确保 diff 更新不会丢失用户输入或导致光标跳动。
- 边界：非受控组件与受控组件之间的转换。

11. 嵌套过深或子节点数量巨大:

- 虽然不是严格的“边界情况”，但 diff 算法的性能在极端复杂的树结构下可能会成为瓶颈。算法本身需要优化以避免不必要的递归或遍历。例如，React 使用启发式算法 (同层比较) 来简化复杂度。

一个健壮的 diff 算法需要在各种组合情况下都能正确、高效地工作。React 和 Vue 都采用了双端比较 (double-ended comparison) 等优化策略来处理列表 diff，以达到 $O(n)$ 的时间复杂度。

3. Webpack的Tree Shaking原理及实现条件

Tree Shaking (树摇) 是一种通过移除 JavaScript 上下文中未引用的代码 (dead-code elimination) 来优化最终打包文件大小的技术。它依赖于 ES2015 (ES6) 模块的静态结构特性。

原理:

1. 静态分析: Tree Shaking 的核心在于 ES6 模块的导入导出是静态的。这意味着在编译时 (Webpack 处理代码时)，就可以确定模块导入了哪些成员，以及哪些成员被导出了。这与 CommonJS 的动态 `require` 不同，CommonJS 的导入可以在运行时确定，难以进行静态分析。
2. 标记依赖: Webpack (或其他支持 Tree Shaking 的打包工具) 会遍历所有模块，分析 `import` 和 `export` 语句。
 - 它会标记出哪些 `export` 的代码被实际 `import` 并在代码中被使用了。
 - 对于未被 `import` 或 `import` 后未被使用的 `export`，它们被认为是“死代码” (dead code)。
3. 代码移除: 在最终生成 bundle 的时候，这些被标记为“死代码”的部分会被移除。这个过程通常由压缩工具 (如 Terser, Webpack 5 默认集成) 在代码压缩阶段完成。Webpack 负责分析和标记，压缩工具负责实际移除。

实现条件:

要使 Tree Shaking 生效, 需要满足以下主要条件:

1. 使用 ES2015 (ES6) 模块语法 (`import` 和 `export`):

- 这是 Tree Shaking 的根本前提。如果代码使用的是 CommonJS (`require` / `module.exports`), Webpack 很难进行有效的静态分析来判断哪些代码是未使用的。
- 需要确保项目中的代码以及依赖的第三方库都尽可能使用 ES6 模块。

2. Webpack 配置:

- `mode: 'production'`: 在生产模式下, Webpack 会自动启用包括 Tree Shaking 在内的多项优化。
- `optimization.usedExports: true` (Webpack 4+): 这个配置 (在 `production` 模式下默认开启) 告诉 Webpack 标记出模块中哪些导出的成员被实际使用。它本身不删除代码, 只是添加 `/* unused harmony export memberName */` 这样的注释。
- `optimization.minimize: true` (并使用支持 Tree Shaking 的压缩器):
 - Webpack 5 默认使用 TerserWebpackPlugin, 它能够识别 `usedExports` 生成的标记并移除死代码。
 - 在更早的版本中, 可能需要手动配置 UglifyJsWebpackPlugin (已不推荐) 或 TerserWebpackPlugin。
- Babel 配置: 如果使用了 Babel, 需要确保 Babel 不会将 ES6 模块转换为 CommonJS 模块。通常通过在 Babel 配置文件 (如 `.babelrc` 或 `babel.config.js`) 中设置 `modules: false` 给 `@babel/preset-env` 来实现:

代码块

```
1  {
2    "presets": [
3      ["@babel/preset-env", { "modules": false }]
4    ]
5  }
```

- 这样 Babel 会保留 ES6 模块语法, 交由 Webpack 处理。

3. 避免副作用 (Side Effects):

- 如果一个模块的导入仅仅是为了其副作用 (例如, 一个 polyfill 模块, 或者一个只是修改了全局变量的模块 `import './utils.js'`), Tree Shaking 可能会错误地认为这个模块没有被使用而将其移除。
- 为了解决这个问题, 可以在 `package.json` 中使用 `sideEffects` 字段:

- `"sideEffects": false`：告诉 Webpack 项目中所有文件都没有副作用，可以大胆地进行 Tree Shaking。
- `"sideEffects": ["/src/some-side-effect.js", "*.css"]`：告诉 Webpack 这些指定的文件或模式匹配的文件有副作用，不应该被 Tree Shaking，即使它们看起来没有被直接导出和使用。
 - 如果模块有副作用，但又想确保它不被移除，需要正确配置 `sideEffects`。

4. 代码本身的书写方式:

- 尽量编写纯函数和无副作用的模块。
- 避免在模块顶层执行有副作用的代码，除非明确标记。
- 例如，如果一个函数调用了全局对象的某个方法，或者修改了全局状态，这可能会被认为是副作用，影响 Tree Shaking 的判断。Terser 等压缩工具会尽可能分析，但不是所有副作用都能被完美检测。

总结来说，Tree Shaking 依赖于 ES6 模块的静态性，通过 Webpack 的分析标记和压缩工具的代码删除，有效地减小了打包体积。正确配置 Webpack、Babel，并注意代码的副作用是确保 Tree Shaking 有效工作的关键。

4. 如何设计一个前端监控系统？

核心目标与思路

设计前端监控系统的核心目标是收集、上报、分析和展示前端应用在用户端的各类数据，以便开发者能够及时发现并定位线上问题、优化应用性能、提升用户体验，并为产品决策提供数据支持。

我的解题思路主要分为以下几个层面：

1. 数据采集 (SDK 端)：明确需要监控哪些数据类型，以及如何在用户浏览器端高效、无感地捕获这些数据。
2. 数据上报：设计可靠、高效的数据传输链路，将采集到的数据发送到服务端。
3. 数据处理与存储 (服务端)：数据到达服务端后如何进行清洗、解析、聚合，并选择合适的存储方案。
4. 数据分析与可视化：如何从海量数据中提取有价值的信息，并通过图表等形式直观展示。
5. 告警机制：当关键指标出现异常时，如何及时通知相关人员。

原理分析

前端监控系统的核心工作原理是在目标前端应用中植入一个轻量级的 JavaScript SDK (Software Development Kit)。这个 SDK 运行在用户浏览器环境中，负责监听和捕获预设的事件和指标。

- 错误监控原理：通过 `window.onerror`、`window.onunhandledrejection` 捕获 JavaScript 运行时错误和 Promise 异步错误。对于资源加载错误，则通过 `window.addEventListener('error', callback, true)` 在捕获阶段监听。对于框架层面的错误（如 React 的 Error Boundaries, Vue 的 errorHandler），SDK 也会进行包裹或利用框架提供的钩子。
- 性能监控原理：利用浏览器提供的 Performance API (如 `PerformanceTiming`、`PerformanceNavigationTiming`、`PerformanceResourceTiming`、`PerformancePaintTiming` 等) 获取页面加载、资源加载、绘制等各个阶段的精确时间戳，计算出 FCP、LCP、FID、CLS、TTI 等核心 Web 指标。
- 行为监控原理：通过对原生 DOM 事件 (如 `click`、`scroll`、`hashchange`) 的监听，或者对特定 API (如 `workspace`，`XMLHttpRequest`，路由跳转方法) 的重写或代理 (AOP 面向切面编程思想)，来捕获用户的交互行为和 API 调用情况。
- 数据上报原理：SDK 将收集到的数据按照预定格式（通常是 JSON）通过 HTTP(S) 请求发送到监控服务器。为了不影响主业务逻辑和页面性能，上报通常采用异步方式，如 `navigator.sendBeacon` (页面卸载时仍能保证发送) 或 `XMLHttpRequest` 异步请求。可能会有采样、合并上报等策略以减少请求量。
- 服务端处理原理：服务端接收数据后，会进行数据清洗 (去除无效数据)、格式化、解析 (如 SourceMap 还原错误堆栈)、聚合计算 (如错误率、性能分位值)，然后存入时序数据库、日志系统或大数据平台。后续通过查询分析系统进行多维度分析和可视化展示。

解答具体的实现方案

一个完整的前端监控系统通常包含以下模块：

1. SDK (Software Development Kit):

◦ 功能模块：

■ 错误捕获模块：

- `try...catch` 包裹关键执行逻辑。
- 重写 `window.onerror`，`window.onunhandledrejection`。
- 监听 `error` 事件捕获资源加载失败。
- 集成框架错误捕获 (React Error Boundary, Vue `errorHandler`)。
- 收集错误信息：错误类型、错误消息、堆栈信息 (SourceMap 还原)、发生时间、用户 UA、页面 URL。

■ 性能数据采集模块：

- 利用 `performance.timing` (旧) 或 `performance.getEntriesByType('navigation')` (新) 获取页面加载时间点 (DNS查询、TCP连接、TTFB、DOM解析、白屏等)。

- 利用 `performance.getEntriesByType('paint')` 获取 FP, FCP。
- 利用 `LargestContentfulPaint` API 获取 LCP。
- 利用 `PerformanceObserver` 监听 `first-input` 获取 FID。
- 利用 `PerformanceObserver` 监听 `layout-shift` 计算 CLS。
- 利用 `performance.getEntriesByType('resource')` 获取资源加载详情。

■ 用户行为追踪模块：

- API 请求监控：通过重写 `XMLHttpRequest` 的 `open` , `send` 方法和 `window.fetch` , 记录请求 URL、方法、状态码、耗时、请求/响应体（脱敏）。
- 路由变化监控：监听 `hashchange` , `popstate` 事件，或重写 `history.pushState` , `history.replaceState` 。
- 用户点击监控：事件委托方式监听 `click` 事件，记录被点击元素的 XPath 或 CSS 选择器。
- 页面留存/跳出：通过 `visibilitychange` 和 `pagehide` 事件。

■ 环境信息收集模块：UA、屏幕分辨率、网络状态 (`navigator.connection`)、地理位置 (需用户授权)。

○ 核心机制：

- 数据格式化与队列：将捕获的数据统一格式化，放入待上报队列。
- 上报策略：定时上报、定量上报、 `navigator.sendBeacon` 兜底上报。
- 采样机制：对于 PV/UV 量大的应用，可配置采样率，避免数据风暴。
- SourceMap 处理：SDK 本身不处理，但收集的错误堆栈需要服务端配合 SourceMap 文件进行还原。
- 轻量化与稳定性：SDK 自身体积要小，不能有性能瓶颈，不能产生错误。

2. 数据上报通道：

- HTTP(S) 接口：服务端提供接收数据的 API 接口。
- 请求方式：通常使用 `POST` 请求， `Content-Type` 为 `application/json` 。
- 数据压缩：SDK 上报前可对数据进行 Gzip 压缩 (如果浏览器支持且传输量大)。
- 安全性：接口可考虑鉴权，防止恶意上报。

3. 服务端 (Data Receiver & Processor)：

- 数据接收网关：高可用、高并发的 HTTP 服务 (如 Nginx + Node.js/Go/Java 应用)。
- 数据缓冲：可选，使用消息队列 (如 Kafka, RabbitMQ) 削峰填谷，解耦数据接收和处理。
- 数据清洗与解析：
 - 校验数据格式和完整性。

- 解析 UserAgent 获取设备、系统、浏览器信息。
- IP 地址解析地理位置。
- SourceMap 还原：根据错误堆栈中的文件名、行号、列号，结合上传的 SourceMap 文件，将混淆压缩后的代码定位到源码位置。这通常需要一个专门的 SourceMap 解析服务。
- 数据聚合与计算：
 - 按小时/天统计错误率、API 成功率。
 - 计算性能指标的平均值、中位数、P75、P90、P95 分位值。
 - 用户行为路径分析等。
- 数据存储：
 - 时序数据库 (Time Series Database)：如 InfluxDB, Prometheus，适合存储性能指标、错误率等带时间戳的序列数据。
 - 日志存储与检索引擎：如 Elasticsearch (ELK Stack)，适合存储错误日志、API 请求日志等，方便全文搜索和聚合分析。
 - 关系型数据库/NoSQL：存储项目信息、用户信息、SourceMap 文件信息等。
- 4. 数据分析与可视化平台 (Dashboard):
 - 图表展示：折线图 (趋势)、柱状图 (分布)、饼图 (占比)、表格 (明细)。
 - 核心指标看板：JS 错误率、影响用户数、TOP 错误、性能概览 (LCP, FID, CLS 平均值/分位数)。
 - 多维度筛选与下钻：按时间、版本、浏览器、操作系统、网络环境、地域等维度筛选数据。
 - 错误详情查看：展示错误堆栈 (还原后)、发生次数、影响用户、最近发生时间、相关环境信息。
 - 用户行为路径可视化 (可选)。
 - 工具选型：Grafana, Kibana, Superset，或自研。
- 5. 告警系统：
 - 配置告警规则：例如，JS 错误率超过阈值、某个版本 LCP 劣化严重、关键 API 连续失败等。
 - 通知渠道：邮件、短信、钉钉、企业微信、Slack 等。
 - 告警聚合与降噪：避免短时间内大量重复告警。

实际面试如何用一段话口语化回答

面试官你好，设计一个前端监控系统，我的思路是这样的：

首先，我们会在前端应用里集成一个轻量级的 JS SDK。这个 SDK 主要负责几件事：一是捕获 JS 错误、资源加载异常以及 API 请求的成功失败；二是采集用户体验相关的性能指标，比如白屏时间、LCP、FID 这些；三是记录用户的关键行为，像是页面跳转、重要点击等。

SDK 收集到这些数据后，会通过 HTTP 请求，用异步或者 `sendBeacon` 的方式，批量、带采样地把数据上报到我们的监控服务器，尽量不影响用户体验。

服务器端接收到数据后，会进行清洗、解析（比如用 SourceMap 还原错误堆栈），然后把结构化的数据存储起来，像错误日志可能存 Elasticsearch，性能数据存时序数据库。

接着，我们会有一个数据分析和可视化平台，通过各种图表展示错误趋势、性能分布、API 质量等，方便我们定位问题和做优化。

最后，还会配置一套告警系统，当错误率飙升或者性能指标严重下降的时候，能及时通知我们。

整个流程就是从数据采集、上报，到后端处理、存储，再到分析展示和告警，形成一个闭环，帮助我们更好地了解和优化前端应用。

5. 实现async/await的generator polyfill

`async/await` 是 ES2017 引入的异步编程语法糖，它基于 Promise 和 Generator 函数。我们可以用 Generator 函数和 Promise 来模拟 `async/await` 的行为。

核心思路：

1. `async` 函数本质上是一个返回 Promise 的函数。
2. `await` 关键字会暂停 `async` 函数的执行，等待其后的 Promise 完成。如果 `await` 后面不是 Promise，会将其包装成一个 resolved 的 Promise。
3. 当 Promise 完成后 (`resolved` 或 `rejected`)，`async` 函数会恢复执行。

Generator Polyfill 实现：

我们需要一个 "执行器" 函数 (通常叫做 `spawn` 或 `asyncToGenerator`)，它接收一个 Generator 函数作为参数。这个执行器函数会：

1. 调用 Generator 函数，得到一个迭代器 (iterator)。
2. 定义一个 `step` 函数，用于驱动迭代器。
3. 在 `step` 函数内部，调用迭代器的 `next(value)` 方法，传入上一个 Promise `resolved` 的值。
4. `next()` 方法返回一个 `{ value, done }` 对象。
 - `value` 是 `yield` 表达式后面的值 (通常是一个 Promise)。
 - `done` 表示 Generator 函数是否执行完毕。
5. 将 `value` 包装成 Promise (如果它还不是 Promise 的话)。
6. 当这个 Promise `resolved` 时，递归调用 `step` 函数，并将 `resolved` 的值传给下一次 `iterator.next()`。

7. 当这个 Promise `rejected` 时，调用迭代器的 `throw(error)` 方法，将错误抛入 Generator 函数内部，这样 `async` 函数内部的 `try...catch` 就可以捕获到。
8. 如果 `done` 为 `true`，说明 `async` 函数执行完毕，此时用最终的返回值 `resolve` 执行器返回的 Promise。

示例代码:

代码块

```
1  function asyncToGenerator(generatorFunc) {
2    // 返回一个新的函数，这个函数就是我们模拟的 async 函数
3    return function(...args) {
4      // 立即执行 generatorFunc，得到一个迭代器
5      const gen = generatorFunc.apply(this, args);
6
7      // 返回一个 Promise，因为 async 函数总是返回 Promise
8      return new Promise((resolve, reject) => {
9        // step 函数用于驱动迭代器的执行
10       function step(nextFn) {
11         let generatorResult;
12         try {
13           // 调用迭代器的 next 或 throw 方法
14           generatorResult = nextFn();
15         } catch (error) {
16           // 如果 generatorFunc 内部直接抛出同步错误
17           return reject(error);
18         }
19
20         const { value, done } = generatorResult;
21
22         if (done) {
23           // 如果迭代器已经完成，用它的最终返回值 resolve Promise
24           return resolve(value);
25         } else {
26           // 如果迭代器未完成，value 通常是一个 Promise
27           // 将 value 确保为一个 Promise
28           return Promise.resolve(value).then(
29             (res) => {
30               // 当 value (Promise) resolved 时，继续下一步
31               // 并将 resolved 的值传给 gen.next()
32               step(() => gen.next(res));
33             },
34             (err) => {
35               // 当 value (Promise) rejected 时，将错误抛回 generator
36               // 这样 async 函数内部的 try/catch 可以捕获
```

```

37         step(() => gen.throw(err));
38     }
39 );
40 }
41 }
42
43 // 启动迭代器
44 step(() => gen.next(undefined));
45 });
46 };
47 }
48
49 // ---- 使用示例 ----
50
51 // 假设有一个返回 Promise 的函数
52 function fetchData(data, delay) {
53     return new Promise(resolve => {
54         setTimeout(() => {
55             console.log(`Workspaceed: ${data}`);
56             resolve(data);
57         }, delay);
58     });
59 }
60
61 function fetchError(message, delay) {
62     return new Promise((resolve, reject) => {
63         setTimeout(() => {
64             console.error(`Error: ${message}`);
65             reject(new Error(message));
66         }, delay);
67     });
68 }
69
70
71 // 使用我们模拟的 async 函数
72 const myAsyncFunction = asyncToGenerator(function* (initialValue) {
73     try {
74         console.log('Starting async operation with:', initialValue);
75         const data1 = yield fetchData(initialValue, 500); // 模拟 await
76         fetchData(initialValue, 500)
77         const data2 = yield fetchData(data1 + ' + data2', 500); // 模拟 await
78         fetchData(...)
79         const data3 = yield fetchData(data2 + ' + data3', 500);
80
81         // 测试错误处理
82         // const errorData = yield fetchError('Something went wrong!', 300);
83         // console.log('This will not be logged if error occurs before this line');

```

```
82
83     return `Final result: ${data3}`;
84 } catch (e) {
85     console.error('Caught error in async function:', e.message);
86     return `Operation failed due to: ${e.message}`;
87 }
```

这个 `asyncToGenerator` 函数就是 Babel 等工具转换 `async/await` 语法时所使用的核心逻辑（Babel 的实现会更复杂和健壮，处理更多边界情况）。

6. 解释 React Hooks 的闭包陷阱及解决方案

React Hooks 的闭包陷阱主要指的是在 `useEffect`，`useCallback`，`useMemo` 等 Hooks 的回调函数中，由于闭包特性，捕获了组件初次渲染或某次特定渲染时的 `state` 或 `props` 值。当这些 `state` 或 `props` 后续发生变化时，回调函数内部依然使用的是旧的、被捕获的值，导致不符合预期的行为。

为什么会发生？

JavaScript 的闭包允许内部函数访问其外部函数作用域中的变量。当一个 Hook 的回调函数（例如 `useEffect` 的 `effect` 函数）被定义时，它会“记住”当时它能访问到的所有 `state` 和 `props`。

- 对于 `useEffect`：如果没有正确设置依赖项数组，或者依赖项数组为空 (`[]`)，`effect` 函数只会在组件挂载时运行一次（或挂载和卸载时）。此时它捕获的是初始的 `state` 和 `props`。之后即使 `state/props` 改变，这个 `effect` 函数（如果它还在被使用，例如在事件监听器或定时器中）内部的 `state/props` 值也不会更新。
- 对于 `useCallback` 和 `useMemo`：它们用于性能优化，通过缓存函数实例或计算结果。如果依赖项数组不正确或缺失，它们返回的函数或值也会捕获旧的 `state/props`。当这个被缓存的函数（例如一个事件处理器）被调用时，它内部引用的 `state` 可能就是过时的。

示例（`useEffect` 陷阱）：

代码块

```
1  import React, { useState, useEffect } from 'react';
2
3  function Counter() {
4      const [count, setCount] = useState(0);
5
6      useEffect(() => {
7          // 这个 effect 只在挂载时运行，因为它没有依赖项（或依赖项为空数组）
8          const intervalId = setInterval(() => {
9              // 这里的 count 是闭包捕获的初始值 0
10             // 所以每次都会是 setCount(0 + 1)，而不是 setCount(当前count + 1)
11             console.log(`Interval tick. Count is (stale): ${count}`);
12             // setCount(count + 1); // 错误的方式，count 永远是 0
13         }, 1000);
14     });
15 }
```

```

14
15     return () => clearInterval(intervalId);
16 }, []); // 空依赖数组，导致闭包陷阱
17
18 // 正确的写法是使用函数式更新，或者将 count 加入依赖项
19 // useEffect(() => {
20 //     const intervalId = setInterval(() => {
21 //         console.log(`Interval tick. Correcting count...`);
22 //         setCount(c => c + 1); // 使用函数式更新，c 是最新的 count
23 //     }, 1000);
24 //     return () => clearInterval(intervalId);
25 // }, []); // 仍然可以空依赖，因为函数式更新不依赖外部 count
26
27 // 或者
28 // useEffect(() => {
29 //     const intervalId = setInterval(() => {
30 //         console.log(`Interval tick. Count from dependency: ${count}`);
31 //         setCount(count + 1); // count 是最新的，因为它是依赖项
32 //     }, 1000);
33 //     return () => clearInterval(intervalId);
34 // }, [count]); // 将 count 加入依赖项
35
36     return (
37         <div>
38             <p>Count: {count}</p>
39             <button onClick={() => setCount(count + 1)}>Increment</button>
40         </div>
41     );
42 }
43
44 export default Counter;

```

解决方案:

1. 正确设置依赖项数组 (Dependency Array):

- 这是最主要的解决方案。在 `useEffect` , `useCallback` , `useMemo` 的第二个参数（依赖项数组）中，列出所有在回调函数内部使用到的、并且可能发生变化的 props 和 state。
- 当数组中的任何一个依赖项发生变化时，React 会重新创建回调函数或重新计算值，这样回调函数就能捕获到最新的 props 和 state。
- ESLint 插件 `eslint-plugin-react-hooks` 中的 `exhaustive-deps` 规则可以帮助自动检测并提示缺失的依赖项。

2. 函数式更新 (Functional Updates) for `useState` :

- 当使用 `setXxx` 函数更新 state 时，可以传递一个函数而不是直接传递新值。这个函数会接收前一个 state 作为参数，并返回新的 state。
- `setCount(prevCount => prevCount + 1);`
- 这种方式不需要在闭包中直接引用当前的 `count` 变量，因此可以避免因为 `count` 是旧值而导致的问题。这在你希望 `useEffect` 的 effect 函数不因为某个 state 的变化而频繁重新执行时特别有用（即依赖项数组可以保持为空或更少）。

3. 使用 `useRef` 存储可变值:

- 如果需要在回调函数中引用最新的某个值，但又不希望这个值的变化触发 `useEffect` 或 `useCallback` 的重新执行，可以将这个值存储在 `useRef` 创建的 ref 对象中。
- Ref 对象 (`ref.current`) 的变化不会触发组件的重新渲染。你可以在 effect 内部读取 `ref.current` 来获取最新的值。
- 这通常用于管理那些不直接参与渲染逻辑，但需要在异步回调中使用的可变数据。

代码块

```
1  const latestCount = useRef(count);
2  useEffect(() => {
3    latestCount.current = count; // 每次渲染都更新 ref
4  });
5
6  useEffect(() => {
7    const intervalId = setInterval(() => {
8      // latestCount.current 总是最新的 count 值
9      console.log('Ticking with latest count:', latestCount.current);
10     }, 1000);
11    return () => clearInterval(intervalId);
12  }, []); // Effect 自身不需要依赖 count
```

4. `useReducer` (对于复杂状态逻辑):

- `useReducer` 返回的 `dispatch` 函数的身份是稳定的，不会在每次渲染时改变。因此，如果 effect 或回调依赖于 `dispatch`，不需要将其加入依赖项数组。这可以简化依赖管理，特别是在状态逻辑复杂时。

总结:

闭包陷阱是 React Hooks 使用中一个常见的挑战，核心在于理解闭包如何捕获变量以及 Hooks 的依赖项数组如何工作。通过正确声明依赖项、使用函数式更新或 `useRef`，可以有效地避免和解决这些问题。ESLint 插件是保持依赖项正确性的好帮手。

7. 设计一个多层级菜单组件，如何实现千万级节点的高性能渲染？

渲染一个包含千万级别节点的多层级菜单组件是一个巨大的性能挑战。直接将所有节点一次性渲染到 DOM 中是不可行的，会导致浏览器卡顿甚至崩溃。核心思路是按需渲染和虚拟化。

关键技术和策略：

1. 虚拟列表/窗口化 (List Virtualization / Windowing):

- 原理: 只渲染当前视口内可见的菜单项，以及视口上下方少量用于缓冲的项。当用户滚动时，动态计算并替换视口内的项。
- 实现:
 - 计算容器的总高度 (基于所有节点的预估高度或实际高度)。
 - 监听滚动事件，根据滚动位置计算当前应该渲染哪些节点。
 - 使用绝对定位将当前可见的节点放置在正确的位置。
 - 需要维护一个内部的扁平化列表或能够快速索引到节点的数据结构。
- 库: `react-window`, `react-virtualized` (React), `vue-virtual-scroller` (Vue) 可以用于实现或提供类似功能。对于菜单这种层级结构，可能需要对这些库进行定制或自研。

2. 懒加载子节点 (Lazy Loading Child Nodes):

- 原理: 对于层级菜单，只在用户展开某个父节点时才去加载和渲染其子节点。
- 实现:
 - 初始渲染时，只渲染顶级节点。
 - 当用户点击展开某个节点时，如果其子节点数据尚未加载，则发起请求获取子节点数据，然后渲染它们。
 - 对于已经加载的子节点，则直接显示/隐藏。
- 这可以显著减少初始渲染的节点数量和数据量。

3. 高效的数据结构与数据处理:

- 扁平化数据 vs 树状数据:
 - 在前端，为了方便查找和渲染，有时会将树状数据拍平，并维护父子关系 (如 `id`, `parentId`，以及节点的深度 `level`)。
 - 拍平的数据结构可能更适合虚拟列表的实现，因为虚拟列表通常处理的是一维列表。
- 快速索引: 需要能够根据节点的 ID 或路径快速定位到节点数据及其在扁平列表中的位置。
- 服务端支持: 如果数据量巨大，前端一次性获取所有节点元数据 (即使只是 ID 和名称) 可能也不现实。可以考虑:
 - 分批加载顶层节点: 如果顶层节点也很多。
 - 服务端搜索/过滤: 允许用户通过搜索快速定位到深层节点，而不是逐级展开。

4. 优化的渲染逻辑:

- `React.memo` / `shouldComponentUpdate`: 对菜单项组件进行优化, 避免不必要的重渲染。
- 事件委托: 将事件监听器 (如点击展开/收起) 绑定在父容器上, 而不是每个菜单项上, 以减少事件监听器的数量。
- CSS 优化: 使用高效的 CSS 选择器, 避免复杂的计算样式。利用 CSS 的 `transform` 和 `opacity` 进行动画, 因为它们通常能利用 GPU 加速。

5. Web Workers (可选, 用于复杂计算):

- 如果节点数据的处理、过滤、排序等操作非常复杂且耗时, 可以考虑将这些计算密集型任务放到 Web Worker 中执行, 避免阻塞主线程, 从而保持 UI 的响应性。

6. 逐步展开与折叠的性能:

- 当展开一个拥有大量直接子节点的父节点时, 即使这些子节点都在视口内, 一次性渲染也可能造成卡顿。可以考虑:
 - 分帧渲染 (Time Slicing / `requestAnimationFrame`): 将子节点的渲染分散到多个帧中进行, 避免单帧任务过长。
 - 只渲染部分子节点 + "加载更多"按钮: 如果一个层级下的子节点实在太多。

7. 状态管理:

- 高效管理每个节点的展开/折叠状态、选中状态等。对于大量节点, 需要避免状态变更导致大规模的组件重渲染。

实现步骤概览:

1. 数据层设计: 确定如何在前端存储和访问这千万级节点的元数据 (至少是ID、名称、是否有子节点、层级)。可能需要一个高效的树形数据结构或拍平的数组配合索引。
2. API 设计: 与后端协商如何分批、懒加载节点数据。API 应支持根据父节点ID获取其直接子节点列表。
3. 组件结构:
 - `VirtualMenuContainer`: 负责滚动、计算可见节点、处理虚拟化逻辑。
 - `MenuItem`: 单个菜单项组件, 负责展示节点内容和处理交互 (展开/收起)。使用 `React.memo` 优化。
4. 渲染逻辑:
 - `VirtualMenuContainer` 内部维护一个表示当前应渲染的节点列表 (通常是扁平的)。
 - 根据滚动位置, 从全量数据 (或已加载数据) 中切片出这部分节点进行渲染。
 - `MenuItem` 组件接收节点数据和层级信息, 渲染时根据层级进行缩进。
5. 交互处理:

- 点击父节点时，触发懒加载（如果子节点未加载），更新数据源，并重新计算虚拟列表的渲染内容。
- 维护节点的展开状态。

对于“千万级”这个量级，纯前端一次性处理所有节点数据本身就是个挑战，即使只是ID和名称。通常这意味着：

- 前端只持有部分已展开或即将展示的节点数据。
- 大量依赖后端的高效查询和数据分页能力。
- 搜索功能变得至关重要，用户不太可能通过逐级展开来找到深层节点。

因此，这里的“渲染”更多的是指高效地展示用户当前关注范围内的数据，并能快速响应用户的导航操作。

8. 前端安全防护方案有哪些？XSS和CSRF的防御手段

前端安全是Web应用开发中至关重要的一环。常见的攻击手段包括 XSS 和 CSRF。

一、跨站脚本攻击 (XSS - Cross-Site Scripting)

XSS 攻击指的是攻击者往 Web 页面里插入恶意 JavaScript 代码，当用户浏览该页之时，嵌入其中 Web 里面的 JavaScript 代码会被执行，从而达到恶意攻击用户的目的。

XSS 类型:

1. 存储型 XSS (Stored XSS): 恶意脚本被永久存储在目标服务器上（如数据库、消息论坛、访客留言等）。当用户请求包含恶意脚本的页面时，脚本从服务器发送给用户并执行。危害最大，影响范围广。
2. 反射型 XSS (Reflected XSS): 恶意脚本作为用户输入的一部分（通常是 URL 参数）提交给服务器，服务器未经充分过滤就将这些脚本反射回用户的浏览器并执行。通常需要诱导用户点击恶意链接。
3. DOM 型 XSS (DOM-based XSS): 攻击不涉及服务器。恶意脚本通过修改页面的 DOM 结构，在客户端被执行。例如，从 URL 的 fragment (#) 中获取数据并直接写入页面。

XSS 防御手段:

1. 输入过滤与输出编码/转义 (Input Filtering & Output Encoding/Escaping):
 - 核心原则: 不信任任何用户输入 (`Never trust user input`)。
 - 输入过滤 (较弱): 对用户输入进行校验，过滤掉或拒绝包含可疑代码的输入。但黑名单策略容易被绕过。
 - 输出编码/转义 (最重要): 当数据需要在浏览器中显示时，根据其插入的上下文进行编码。
 - HTML 内容中: 将特殊字符如 `<` 替换为 `<` , `>` 替换为 `>` , `&` 替换为 `&` , `"` 替换为 `"` , `'` 替换为 `'` 或 `'` 。

- HTML 属性中: 同样进行转义, 特别是当属性值可能闭合属性或创建新属性时 (例如 `href="javascript:..."`)。对引号敏感的属性需要特别注意。
- JavaScript 上下文中: 当数据要插入到 `<script>` 标签内或事件处理器中时, 情况更复杂。最好避免将用户数据直接拼接到 JS 代码中。如果必须, 需要进行非常严格的 JavaScript 编码 (如 JSON 序列化后在 JS 中解析, 或使用专门的 JS 编码库)。对 `\`、单引号、双引号、换行符等进行转义。
- CSS 上下文中: 避免用户数据直接写入 `<style>` 标签或 `style` 属性中, 因为 CSS 也有执行代码的方式 (如 `expression()`, `url('javascript:...')`)。如果必须, 严格过滤和编码。
- URL 上下文中: 对 URL 参数进行 `encodeURIComponent`。

2. 内容安全策略 (CSP - Content Security Policy):

- 通过 HTTP 头部 (`Content-Security-Policy`) 告诉浏览器哪些来源的资源是可信的, 从而限制浏览器加载和执行恶意资源。
- 可以限制: `script-src` (JS来源), `style-src` (CSS来源), `img-src` (图片来源), `default-src` (默认来源) 等。
- 可以禁止 `inline-script` 和 `eval()`。
- 例如: `Content-Security-Policy: default-src 'self'; script-src 'self' https://trusted.cdn.com;`

3. HttpOnly Cookie:

- 将敏感 Cookie (如 session ID) 标记为 `HttpOnly`, 这样 JavaScript 就无法通过 `document.cookie` API 读取到它, 从而防止 XSS 窃取 Cookie。

4. 设置 X-XSS-Protection 响应头 (较旧, CSP 更佳):

- `X-XSS-Protection: 1; mode=block` 可以让浏览器启用内置的 XSS 过滤器。但现代浏览器更推荐使用 CSP。部分浏览器已弃用此头部。

5. 富文本编辑器特别处理:

- 富文本编辑器允许用户输入 HTML。需要使用成熟的富文本库, 并配置其白名单机制, 严格过滤掉危险的标签和属性 (如 `<script>`, `<iframe>`, `onerror`, `onclick` 等)。例如 `DOMPurify` 库。

6. 框架自带的防御机制:

- 现代前端框架 (React, Vue, Angular) 默认会对插入到 DOM 中的动态内容进行转义, 从而在一定程度上防御 XSS。但当使用 `dangerouslySetInnerHTML` (React) 或 `v-html` (Vue) 等直接插入 HTML 的功能时, 需要开发者自行确保内容安全。

二、跨站请求伪造 (CSRF - Cross-Site Request Forgery)

CSRF 攻击指的是攻击者盗用了你的身份，以你的名义发送恶意请求。例如，用户登录了银行网站 A，然后在未退出的情况下访问了恶意网站 B，恶意网站 B 可能包含一个自动向银行网站 A 发送转账请求的表单或脚本。由于用户的浏览器中存有银行网站 A 的 Cookie，这个请求会携带 Cookie 发送，银行网站 A 的服务器会误以为是用户本人操作。

CSRF 防御手段:

1. 验证 HTTP Referer 字段:

- 在 HTTP 请求头中，`Referer` 字段记录了该请求的来源地址。服务器可以检查 `Referer` 是否来自受信任的域名。
- 缺点: `Referer` 可以被用户或某些安全软件禁止发送；某些情况下（如从 HTTPS 跳转到 HTTP）浏览器不发送 `Referer`；攻击者在某些情况下可能伪造 `Referer`（虽然较难）。

2. 使用 CSRF Token (Anti-CSRF Token):

- 最常用且有效的方案。
- 原理:
 - i. 用户登录后，服务器为该用户生成一个唯一的、随机的 Token (CSRF Token)。
 - ii. 服务器将此 Token 嵌入到前端页面的表单中 (作为隐藏字段) 或者通过 API 返回给前端。
 - iii. 当用户提交表单或发送敏感操作请求 (如 POST, PUT, DELETE) 时，前端将此 Token 一同提交给服务器 (可以放在请求参数、请求体或自定义请求头中)。
 - iv. 服务器在收到请求后，验证提交的 Token 是否与之前为该用户会话存储的 Token 一致。
 - v. 如果 Token 不一致或缺失，服务器拒绝该请求。
- 要点: Token 必须是随机且不可预测的；Token 应该与用户会话关联；Token 应该有过期时间。
- 攻击者无法获取到用户在目标网站的 CSRF Token，因此伪造的请求无法通过验证。

3. SameSite Cookie 属性:

- `SameSite` 属性允许服务器声明 Cookie 是否应该随跨站请求一起发送。
- `Strict`: 完全禁止第三方 Cookie，Cookie 只在当前站点发送。
- `Lax`: 允许部分第三方 GET 请求携带 Cookie (例如导航链接跳转)。POST、PUT、DELETE 等不安全方法和 AJAX 请求不会携带。这是目前很多浏览器的默认值。
- `None`: 允许跨站请求携带 Cookie，但必须同时设置 `Secure` 属性 (即 Cookie 只能通过 HTTPS 发送)。
- 将敏感操作相关的 Cookie (如 session Cookie) 设置为 `SameSite=Lax` 或 `SameSite=Strict` 可以有效防御 CSRF。

4. 对敏感操作增加二次验证:

- 对于非常关键的操作 (如修改密码、支付、删除重要数据)，除了 CSRF Token 外，还可以要求用户输入密码、短信验证码、指纹等进行二次确认。

5. 统一校验 HTTP Method:

- 对于修改数据的操作，严格使用 POST/PUT/DELETE 方法，避免使用 GET 方法进行写操作，因为 GET 请求更容易被嵌入到 `<img>` 标签或链接中被动触发。

总结:

- XSS 防御核心: “输入过滤，输出转义/编码”，并配合 CSP。
- CSRF 防御核心: “CSRF Token” 和 “SameSite Cookie”。

前端安全是一个持续对抗的过程，需要开发者保持警惕，并综合运用多种防御策略。

9. 微前端方案qiankun的沙箱隔离原理

`qiankun` 是一个流行的微前端框架，其核心能力之一就是为子应用提供沙箱隔离，以确保子应用之间的样式不冲突、全局 JavaScript 环境不互相污染。`qiankun` 的沙箱主要分为 JS 沙箱 和 CSS 沙箱。

一、JS 沙箱 (JavaScript Sandbox)

JS 沙箱的目标是使得每个子应用在运行时仿佛拥有一个独立的全局环境 (`window` 对象)，对全局变量的修改 (如 `window.a = 1`)、事件监听 (`window.addEventListener`) 等操作被隔离在当前子应用内部，不会影响主应用或其他子应用。

`qiankun` 提供了多种 JS 沙箱实现，以适应不同浏览器和场景：

1. SnapshotSandbox (快照沙箱 - 旧版本或兼容性方案):

- 原理:
 - 在子应用激活 (mount) 前，拍摄当前 `window` 对象的一个快照 (遍历 `window` 上的属性)。
 - 子应用运行时，对 `window` 的所有修改都会记录下来。
 - 在子应用卸载 (unmount) 时，将 `window` 对象恢复到进入子应用前的快照状态，并将子应用期间的修改应用到其自身的“沙箱环境”记录中。当子应用再次激活时，会先恢复主应用的 `window`，然后将子应用上次的修改重新应用到 `window` 上。
- 优点: 兼容性好，能在不支持 `Proxy` 的旧浏览器中运行。
- 缺点:
 - 性能开销较大，因为需要遍历 `window` 对象。
 - 无法同时运行多个子应用实例 (因为它们共享同一个 `window`，只是在切换时恢复状态)。
 - 某些无法通过遍历枚举的 `window` 属性可能无法正确隔离 (如 `iframe` 内的 `window` 属性)。

2. LegacySandbox (单例代理沙箱 - 基于 Proxy, qiankun 早期版本):

- 原理:
 - 在子应用激活时, 使用 Proxy 代理 window 对象。
 - 当子应用试图访问或修改 window 上的属性时:
 - 读取 (get): 优先从沙箱自身的“伪 window 对象” (记录子应用修改的 K-V 集合) 中查找。如果找不到, 则从真实的 window 对象上查找。
 - 修改 (set): 所有修改都记录在沙箱自身的“伪 window 对象”中, 不直接修改真实的 window 对象。
 - 删除 (deleteProperty): 同样操作于“伪 window 对象”。
 - 当子应用卸载时, 沙箱状态保留。再次激活时, 继续使用这个代理。
- 优点: 比快照沙箱性能好, 避免了 window 属性的遍历和恢复。
- 缺点: 仍然是单例的, 不支持同时运行多个子应用实例共享同一个沙箱类型。Proxy 有兼容性问题 (IE 不支持)。

3. ProxySandbox (多例代理沙箱 - 基于 Proxy, qiankun 推荐):

- 原理: 这是目前 qiankun 推荐的默认 JS 沙箱 (如果浏览器支持 Proxy)。
 - 为每个子应用实例创建一个独立的“伪 window 对象” (fakeWindow)。
 - 使用 Proxy 代理这个 fakeWindow。当子应用访问全局变量时:
 - 读取 (get): 优先从 fakeWindow 自身查找。如果找不到, 则从外部真实的 window 对象上查找。
 - 修改 (set): 所有对全局变量的修改都只发生在 fakeWindow 上。
 - hasOwnProperty 等检查: 也是先查 fakeWindow, 再查真实 window。
 - 这样, 每个子应用都认为自己在操作全局 window, 但实际上所有操作都被 Proxy 拦截并导向了其私有的 fakeWindow。
- 优点: 支持同时运行多个子应用实例, 每个实例都有自己独立的 JS 运行环境。隔离性强。
- 缺点: 依赖 Proxy, 在不支持 Proxy 的浏览器中需要降级。

4. IE 降级方案 (基于 iframe 或 with 的模拟 - 较少直接提及, 但原理类似):

- 在不支持 Proxy 的极端情况下, qiankun 可能会采用更传统的隔离方式, 或者开发者需要自行处理。例如, 在非常老的方案中, iframe 因其天然的隔离性被用于 JS 沙箱, 但通信复杂。with 语句也曾被考虑过, 但因其性能问题和严格模式下的限制而不推荐。qiankun 的快照沙箱是更实用的降级方案。

JS 沙箱对 window.addEventListener 等的特殊处理:

沙箱还需要正确处理 setTimeout, setInterval, addEventListener 等全局 API。通常的做法是:

- 子应用调用 `window.addEventListener` 时，沙箱会记录下这些事件监听器。
- 当子应用卸载时，沙箱会自动移除这些监听器，避免内存泄漏和副作用。

二、CSS 沙箱 (Style Sandbox)

CSS 沙箱的目标是防止子应用的样式泄露到主应用或其他子应用，也防止主应用的样式污染子应用。

`qiankun` 主要通过以下两种方式实现 CSS 隔离：

1. Shadow DOM (影子 DOM):

- 原理: 将子应用的整个 DOM 结构渲染到一个 Shadow Root 中。Shadow DOM 具有天然的样式隔离特性，内部的样式不会影响到外部，外部的样式也不会影响到内部（除非使用特定选择器如 `::part` 或 CSS 自定义属性穿透）。
- 优点: 隔离性最强，是 W3C 标准方案。
- 缺点:
 - 有一定的兼容性问题 (虽然现代浏览器支持良好)。
 - 某些依赖于全局 DOM 结构或事件冒泡的库可能在 Shadow DOM 中工作不正常。
 - 弹窗、全局提示等需要挂载到 `document.body` 的组件，如果其样式依赖于子应用在 Shadow DOM 内的上下文，可能会出现样式问题。

2. Scoped CSS / Runtime Style Scoping (动态样式表):

- 原理: 当 Shadow DOM 不可用或不适用时，`qiankun` 会采用这种方案。
 - 在子应用加载时，会遍历其所有 `<style>` 和 `<link rel="stylesheet">` 标签。
 - 重写这些样式规则，为其添加特定的作用域选择器 (例如，基于子应用的唯一 ID，给所有选择器加上前缀如 `div[data-app-name="app1"] .my-class { ... }`)。
 - 或者，更常见的是，将子应用的样式表内容提取出来，动态创建 `<style>` 标签插入到主应用的 `<head>` 中。在子应用卸载时，移除这些动态添加的 `<style>` 标签。
 - 为了防止主应用样式污染子应用，`qiankun` 可能会尝试为主应用的样式也加上类似的作用域限制，或者在子应用激活时，暂时禁用一部分可能冲突的主应用样式（这种做法较复杂且风险高）。
 - `qiankun` 还可以通过在子应用的根元素上添加一个特定的属性选择器，然后动态修改子应用的所有 CSS 规则，使其只对带有这个属性选择器的元素生效。
- 优点: 兼容性好。
- 缺点:
 - 隔离性不如 Shadow DOM 完美，复杂的 CSS 规则或 JS 动态添加的内联样式可能难以完全隔离。
 - 重写 CSS 规则有一定性能开销。

`qiankun` 通常会优先尝试使用 `experimentalStyleIsolation` (基于 Shadow DOM) 或根据配置和浏览器能力选择合适的 CSS 隔离方案。

总结来说, `qiankun` 的沙箱机制通过结合 `Proxy` (或快照) 实现 JS 环境隔离, 以及 Shadow DOM (或动态样式表) 实现 CSS 隔离, 为微前端架构提供了重要的安全和稳定保障。

10. 服务端渲染hydration过程可能出现的问题

服务端渲染 (SSR - Server-Side Rendering) 的 Hydration (注水/激活) 过程指的是在浏览器端, JavaScript 接管由服务器发送过来的静态 HTML 内容, 并使其变为可交互的动态应用的过程。这个过程虽然能带来更好的首屏性能和 SEO, 但也可能遇到一些问题:

1. Checksum Mismatch / Hydration Mismatch (HTML 内容不匹配):

- 问题: 服务器渲染生成的 HTML 结构或内容与客户端首次渲染 (在 hydration 之前, 框架通常会进行一次虚拟 DOM 的构建和对比) 期望的 HTML 结构或内容不一致。
- 原因:
 - 动态数据差异: 服务端和客户端获取的初始化数据不一致 (例如, 时间戳、随机数、依赖用户特性但服务端无法获取的数据如 `window.innerWidth` 等)。
 - 条件渲染差异: 服务端和客户端执行条件渲染逻辑 (如 `if/else`, `v-if/v-show`) 的结果不同。
 - 第三方库差异: 某些库在服务端和客户端行为不一致, 或产生了不同的 DOM 结构。
 - 浏览器扩展或用户脚本: 浏览器插件修改了服务端返回的 HTML。
 - 代码逻辑错误: 服务端和客户端的组件渲染逻辑本身存在不一致。
 - 环境差异: 不同的 `Date().toLocaleString()` 结果, 时区问题。
 - HTML 规范性问题: 服务器生成的 HTML 不规范 (如 `<table>` 内直接放 `<div>`), 浏览器解析后DOM结构与预期不符。
- 影响: React 等框架在检测到 mismatch 时, 通常会放弃 hydration, 并重新在客户端完整渲染整个应用或受影响的子树, 这会失去 SSR 的部分性能优势, 可能导致页面闪烁或用户体验下降。Vue 则可能在开发模式下给出警告, 生产模式下尝试“迁就”不一致。
- 排查: 关注控制台的警告信息, 仔细对比服务端 HTML 和客户端期望的 VDOM 结构。

2. 客户端独有逻辑 (Client-Only Components / Logic):

- 问题: 某些组件或逻辑只能在客户端运行 (例如, 依赖 `window`, `document` 等浏览器API的图表库、地图库、动画库, 或者需要用户交互后才加载的组件)。如果在服务端尝试渲染它们, 可能会报错或产生不正确的输出。
- 解决:

- 使用框架提供的机制来标记这些组件为“客户端独有”。例如，Next.js 的 `dynamic import` 配合 `ssr: false`，Nuxt.js 的 `<client-only>` 组件。
- 在组件内部通过 `useEffect` (React) 或 `mounted` (Vue) 等生命周期钩子来执行客户端独有的初始化操作，确保相关代码只在浏览器端执行。

3. 状态不一致 (State Inconsistency):

- 问题: 服务端渲染时应用有一个初始状态，这个状态需要正确地传递到客户端，以便 hydration 时客户端应用能够从同样的状态开始。如果状态传递错误或丢失，客户端应用的状态可能与服务端渲染的内容不匹配。
- 解决:
 - 将服务端计算好的初始状态序列化后嵌入到 HTML 页面中 (通常放在 `<script>` 标签内，如 `window.__INITIAL_STATE__ = ...;`)。
 - 客户端在初始化 store (如 Redux, Vuex, Pinia) 时，从这个全局变量中读取并应用初始状态。

4. 事件监听器和交互性延迟:

- 问题: 虽然 HTML 内容很快到达浏览器，但与之关联的 JavaScript (用于 hydration 和添加事件监听器) 可能需要一段时间才能下载、解析和执行完毕。在此期间，页面看起来是完整的，但用户交互 (如点击按钮) 可能没有反应。
- 影响: 导致用户感知到页面“假死”。
- 解决:
 - 优化 JavaScript 包大小和执行效率。
 - 使用代码分割，优先加载并执行用于 hydration 关键部分的代码。
 - 考虑渐进式 hydration 或部分 hydration (Partial Hydration / Islands Architecture)，只激活页面上真正需要交互的部分。

5. 双重数据获取 (Double Data Fetching):

- 问题: 如果处理不当，组件可能会在服务端获取一次数据用于渲染，然后在客户端 hydration 时又获取一次相同的数据。
- 影响: 浪费资源，可能导致数据不一致或页面内容变化。
- 解决: 确保客户端能够复用服务端获取的数据 (通过上面提到的状态传递机制)，并在客户端有条件地跳过初始数据获取。

6. 样式问题 (Style Flashing / FOUC - Flash of Unstyled Content):

- 问题: 如果 CSS 加载和应用不当，用户可能会先看到无样式的 HTML 内容，然后样式才应用，造成闪烁。对于 CSS-in-JS 方案，需要确保服务端能正确提取并内联关键 CSS。
- 解决:

- 将关键 CSS 内联到 HTML 头部。
- 确保 CSS 文件的加载优先级。
- CSS-in-JS 库通常有专门的 SSR 支持方案来收集和注入样式。

7. 内存泄漏:

- 问题: 在服务端渲染大量页面时, 如果代码中存在内存泄漏 (例如, 全局变量持续增长, 事件监听器未移除等), 服务器的内存消耗会不断增加, 最终可能导致崩溃。
- 解决: 编写健壮的代码, 注意清理副作用, 特别是在 Node.js 环境下。

8. 路由和代码分割:

- 问题: 需要确保服务端能够根据请求的 URL 正确渲染对应的页面, 并且客户端在 hydration 后能接管路由。代码分割也需要在服务端和客户端正确协调。
- 解决: 使用支持 SSR 的路由库, 并正确配置代码分割点, 确保服务端能找到并渲染正确的组件, 客户端能加载对应的 chunk。

处理好这些 hydration 问题是成功实施 SSR 的关键, 需要细致的配置和对框架 SSR 机制的深入理解。

11. 实现一个支持撤销重做的状态管理系统

实现一个支持撤销 (Undo) 和重做 (Redo) 功能的状态管理系统, 通常需要记录状态的变更历史。这里提供一个基于命令模式 (Command Pattern) 的简化思路, 也可以直接存储状态快照。

核心思路:

1. 状态 (State): 应用的核心数据。

2. 历史栈 (History Stacks):

- `undoStack`: 存储能够“撤销”到过去状态的操作或状态快照。
- `redoStack`: 存储能够“重做”已撤销操作或状态快照。

3. 操作/命令 (Action/Command) 或状态快照:

- 命令模式: 每个改变状态的操作被封装成一个命令对象, 该对象包含执行 (`execute`) 和撤销 (`undo`) 该方法。当执行一个命令时, 将其推入 `undoStack`。撤销时, 从 `undoStack` 弹出命令并执行其 `undo` 方法, 然后将该命令推入 `redoStack`。重做时, 从 `redoStack` 弹出命令并执行其 `execute` 方法, 再将其推回 `undoStack`。
- 状态快照模式: 每次状态变更后, 将变更前的状态 (或整个状态) 存入 `undoStack`。撤销时, 将当前状态存入 `redoStack`, 并从 `undoStack` 中取出上一个状态作为当前状态。重做时, 将当前状态存入 `undoStack`, 并从 `redoStack` 中取出状态作为当前状态。这种方式更简单, 但如果状态对象很大, 可能会有性能和内存开销。

基于状态快照的简易实现:

代码块

```
1  class UndoRedoState {
2    constructor(initialState) {
3      this.undoStack = []; // 存储过去的状态快照
4      this.redoStack = []; // 存储已撤销的状态快照
5      this.currentState = this._clone(initialState); // 确保初始状态不被直接修改
6    }
7
8    _clone(state) {
9      // 简单的深拷贝，对于复杂对象可能需要更完善的深拷贝函数
10     // 对于不可变数据结构，这里可以直接赋值
11     if (typeof state !== 'object' || state === null) {
12       return state;
13     }
14     try {
15       return JSON.parse(JSON.stringify(state));
16     } catch (e) {
17       // 对于包含函数、Date、RegExp 等 JSON 不支持的，需要自定义克隆
18       console.warn("Cloning failed for complex object, using shallow copy as
19 fallback or implement deepClone.", state);
20       return {...state}; // 极简回退
21     }
22   }
23
24   setState(newStateProducer) {
25     // newStateProducer 可以是一个新状态，或者一个接收旧状态返回新状态的函数
26     const oldState = this._clone(this.currentState); // 保存当前状态的快照
27
28     let newState;
29     if (typeof newStateProducer === 'function') {
30       newState = newStateProducer(this._clone(this.currentState)); // 将当前状态
31       // 的克隆传给生产者函数
32     } else {
33       newState = newStateProducer;
34     }
35
36     // 检查状态是否真的改变了，避免不必要的历史记录
37     // (对于复杂对象，这里的比较可能需要深度比较)
38     if (JSON.stringify(oldState) === JSON.stringify(newState)) {
39       // console.log("State did not change.");
40       return;
41     }
42   }
43 }
```

```

42     this.undoStack.push(oldState);
43     this.currentState = this._clone(newState); // 更新当前状态
44     this.redoStack = []; // 任何新的状态变更都会清空重做栈
45
46     // 可选：限制历史栈的大小
47     // if (this.undoStack.length > MAX_HISTORY_SIZE) {
48     //     this.undoStack.shift();
49     // }
50
51     console.log("State updated:", this.currentState);
52     this._logStacks();
53 }
54
55 undo() {
56     if (this.undoStack.length === 0) {
57         console.log("Nothing to undo.");
58         return this.currentState;
59     }
60
61     const previousState = this.undoStack.pop();
62     this.redoStack.push(this._clone(this.currentState)); // 当前状态存入重做栈
63     this.currentState = previousState; // 恢复到上一个状态
64
65     console.log("State undone:", this.currentState);
66     this._logStacks();
67     return this.currentState;
68 }
69
70 redo() {
71     if (this.redoStack.length === 0) {
72         console.log("Nothing to redo.");
73         return this.currentState;
74     }
75
76     const nextState = this.redoStack.pop();
77     this.undoStack.push(this._clone(this.currentState)); // 当前状态存入撤销栈
78     this.currentState = nextState; // 恢复到下一个状态
79
80     console.log("State redone:", this.currentState);
81     this._logStacks();
82     return this.currentState;
83 }
84
85 getCurrentState() {
86     return this._clone(this.currentState); // 返回当前状态的副本

```

关键点与改进方向:

1. 深拷贝 (`_clone`): 对于复杂的状态对象 (包含 Date, RegExp, Map, Set, 函数, 或循环引用), `JSON.parse(JSON.stringify(state))` 是不够的。需要一个更健壮的深拷贝函数 (如 lodash 的 `_cloneDeep`), 或者使用不可变数据结构 (Immutability)。
2. 不可变数据结构 (Immutability): 使用像 Immer, Immutable.js 这样的库可以简化状态管理和快照。它们通过结构共享来优化性能和内存使用, 并且使得状态变更的检测更高效。如果使用 Immer, `setState` 内部的修改会更简洁。
3. 性能与内存: 对于非常大的状态对象或非常频繁的更新, 存储完整的状态快照可能会消耗大量内存。
 - 差异存储 (Diffing): 可以只存储状态之间的差异 (patches), 而不是完整的快照。撤销/重做时, 应用这些 patch。这需要实现 diff/patch 算法。
 - 限制历史栈大小: 可以设置 `undoStack` 和 `redoStack` 的最大长度, 超过则丢弃最旧的记录。
4. 命令模式 (Command Pattern):
 - 如果状态变更逻辑比较复杂, 或者希望对操作本身进行更细致的管理 (例如, 合并连续的微小操作), 命令模式会更灵活。
 - 每个命令对象需要实现 `execute()` 和 `undo()`。`undo()` 方法需要知道如何逆转 `execute()` 的效果。
 - 例如, 一个 `ChangeValueCommand` 可能存储旧值和新值, `execute` 应用新值, `undo` 恢复旧值。
5. 异步操作: 如果状态变更是异步的, 需要在异步操作完成后才记录历史。
6. 批量操作/事务: 有时希望将多个连续的操作视为一个原子操作, 可以一起撤销/重做。这可以通过将一组命令打包成一个宏命令 (MacroCommand) 或引入事务边界来实现。
7. 集成到 UI 框架: 在 React/Vue 等框架中, 可以将这个状态管理器与组件状态或全局状态库 (Redux, Vuex, Zustand, Pinia) 结合。例如, 可以创建一个自定义 Hook (React) 或一个插件/Store 模块 (Vue)。

这个简易实现提供了一个基本框架, 实际应用中需要根据具体需求选择合适的策略并处理更多细节。对于复杂的应用, 推荐使用现有的、经过良好测试的状态管理库, 它们通常内置或可以方便地集成撤销/重做功能。